

SemEval 2026 Task 9: System Description Paper

Kevin McMahon

University of Colorado, Boulder
kevin.mcmahon-1@colorado.edu

Caleb Kumar

University of Colorado, Boulder
caleb.kumar@colorado.edu

Abstract

The SemEval-2026 Task 9 challenge focuses on detecting and categorizing polarized discourse across 22 languages, capturing how group-based hostility and division manifest in online text. Primarily across all 3 of the sub-tasks, we fine-tuned different RoBERTA models as multilingual encoders and applies a sigmoid-based multi-label classification head. We jointly train across five languages: English, Spanish, Arabic, German, and Chinese, using curated pre-processing and a threshold-optimized prediction strategy to improve recall for minority label types. Our results demonstrate strong multilingual generalization with notable variation across label types. Overall, our findings show that multilingual transformer models provide a strong foundation for multi-label polarization classification, though challenges remain for broad, heterogeneous target categories.

1 Introduction

Online discourse increasingly reflects the social, political, and cultural divisions that shape contemporary society. As conversations move across languages and platforms, expressions of polarization like vilification, group based hostility, and ideological reinforcement become more difficult to analyze at scale. Automatic systems that can detect and categorize polarized content are therefore essential for supporting content moderation, understanding public opinion, and studying harmful discourse across diverse linguistic communities.

SemEval-2026 Task 9, addresses this challenge by introducing a large, multilingual dataset spanning 22 languages and a range of social and geopolitical contexts. The task is divided into three subtasks: (1) binary detection of whether a text expresses polarization,

(2) multi-label classification of the target types of polarization, and (3) manifestation identification. Together, these subtasks provide a comprehensive framework for modeling polarized discourse in a multilingual setting.

2 Tasks Overview

Task 9 consists of 3 different subtasks, all focused on modeling different aspects and features of polarization across many different languages. Subtask 1 is a binary classification task, determining whether a given text has any form of polarization in it. Subtask 2 adds another layer to subtask 1 by adding multi-label classification of different types of polarization. These categories are: political, racial/ethnic, religious, gender/sexual, and other. Subtask 3 then rounds out the problem by adding in a real-world context. It does this by adding real-world events related to polarizing messages/texts from an event ontology. All 3 of these subtasks move from broad to fine-grained categorization, testing Large Language Model’s abilities to provide a comprehensive analysis of polarization across multiple languages and contexts.

3 Data

A data repository for each subtasks was provided by the organizers of SemEval-2026 Task 9. This data can only be accessed through CodaBench by registering for the competition. The dataset is divided between the 3 subtasks (*Polarization Detection*, *Polarization Type Classification*, *Polarization Manifestation Identification*). Currently the challenge is in the Developmental Phase, which means

that we are given the train data which obviously contains labels and then a development (validation) dataset that does not have labels which we can test our models on by submitting submissions to the challenge here on Codabench, <https://www.codabench.org/competitions/10522/>.

3.1 Data by Language

Each subtask will have 22 languages to use or train from, though at the time of this project only 13 were available. The compute power to train a model using 13 languages is expensive, and so for this project each subtask is fine-tuned using 5, *English, Spanish, German, Arabic and Chinese*. This was reasoned to be a strong subset of languages to choose from as they all have some variation across multiple dimensions. One could only argue that Spanish and English are common as they both have Latin influence (Spanish more so than English as it is a romance language), though ultimately they share *Subject-Verb-Object (SVO)* word ordering. German differs in word ordering as it implements V2 (Verb-second) word ordering, though ultimately the key difference which causes the language models to work harder is the extent of compounding. Compounding affects the tokenization and can make it harder for the model to identify certain tokens that may be more polarizing when they are glued between 3 or more words rather than 2 which we see in english. The primary motivation for choosing Arabic and Chinese to train our models with was primarily due to the syntactical differences. Though as mentioned often in class when on the topic of tokenization, Chinese introduces many challenges since there are no spaces between words. This leads to a dense representation which makes ultimately adds another dimension to the model as now it must also learn segmentation. Arabic was the most interesting of languages picked by the team. The rest of the languages read left-to-right, though Arabic reads right-to-left. Though given we were using encoders which can look at tokens both left and right of the current token, not just previous tokens as in *decoders* so this may not pose too much of a

challenge in the models trained given they are all encoders. Lastly about Arabic, the character script changes given the position of the word so the same word has different representations given position.

3.2 Exploratory Data Analysis

The data provided by the organizers of this challenge was clean. No formal data cleaning went into the process as it the data had no missing values and was well formatted. Additionally being a classification task, we did not have to worry about outliers or irregular values in the data and ultimately just the distribution by language and the different labels.

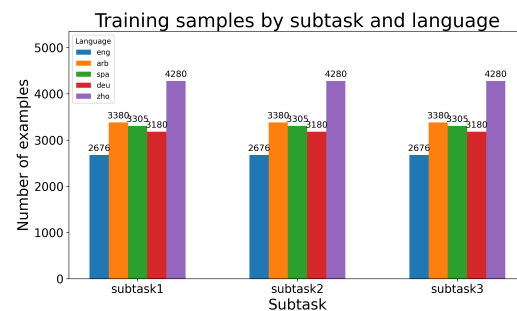


Figure 1: Number of training samples by language for each subtask.

Though talked about in class most often (especially when discussing *Machine Translation*), we have the least number of training samples in English than any other language. Though the number of english training samples is low, Chinese has the most training samples, 4,280, which is almost 60% more than the number of english samples. We are not certain whether this disparity in the number of samples per language will make a difference in overall performance since the number of training samples by language is constant across all subtasks. Overall though this is assumed to not have made a considerable difference in overall performance because for each of the subtasks we are fine-tuning models from Hugging Face that have already been trained on millions of training samples - so hopefully presumably the difference between samples by language being on the order of a thousand should not make a difference.

Label distribution is a similar story - not

perfect, but should be good enough given we are fine tuning massive models models.

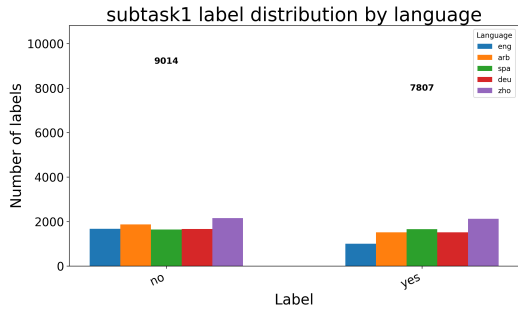


Figure 2: Binary label distribution for subtask 2 by language.

There is just over 15% more negative polarization samples compared to positive samples across the chosen languages. The extent to which this causes issues is explored more in-depth later in the Subtask 1 section as the same model is trained with the provided data and then the team tests the affects of data augmentation techniques are tested. The team was interested in the affects of simple data augmentation on model performance.

As for the multi-class classification tasks the data is not as well distributed, especially subtask 2. As for each task, there are classes which definitely lack data especially for english as it is the language with the least amount of overall samples. Here is distributions of classes for both of the multi-class classification tasks below.

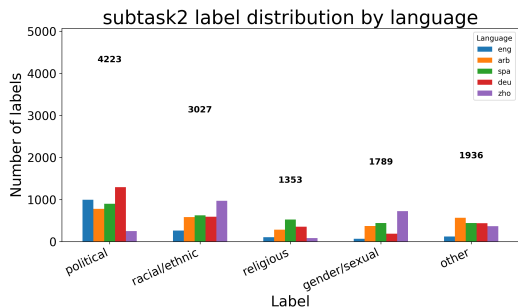


Figure 3: Label distribution for subtask 2 by language.

As for subtask 2, the trends in number of samples by class is uniform for all languages except for Chinese where the 'racial/ethnic' class was most prominent instead of 'political' class. The difference in samples by class is most extreme for English and Chinese.

Label	ARB	DEU	ENG	SPA	ZHO
Total Examples	3380	3180	2676	3305	4280
Political	780	1295	996	901	251
Racial/Ethnic	583	589	264	623	968
Religious	283	354	106	525	85
Gender/Sexual	369	187	67	443	723
Other	565	439	121	443	368

Table 1: Subtask 2 label counts by language.

Above is a table containing the total number of samples by class for each language. Note that the counts for each language do not add up to the total example count as we had many examples which did not fit into any class. Ultimately, based on these counts the most difficult classes to predict will be: (1) gender/sexual english polarization (2.5% of english examples fall into this class) and (2) religious Chinese polarization (1.9% of Chinese examples fall into this class).

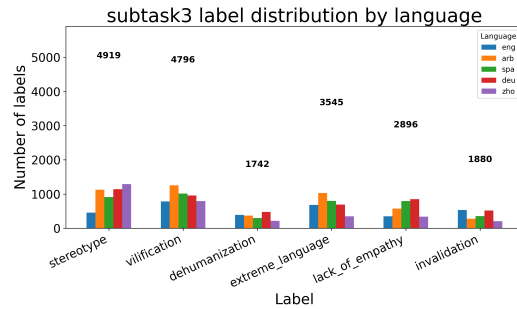


Figure 4: Label distribution for subtask 3 by language.

Similar to subtask 2, the class distribution is not equal. The counts are below.

Label	ARB	DEU	ENG	SPA	ZHO
Total Examples	3380	3180	2676	3305	4280
Stereotype	1127	1140	457	908	1287
Vilification	1256	956	781	1011	792
Dehumanization	370	475	387	295	215
Extreme Language	1027	692	679	799	348
Lack of Empathy	575	848	345	791	337
Invalidation	274	517	533	351	205

Table 2: Subtask 3 label counts by language.

The label counts are not as extreme as subtask 2. The most difficult class to predict statistically should be Chinese Invalidation with less than 5% of Chinese samples falling into that class.

In conclusion, the data is clean so pre-processing was unnecessary. Though the unequal class distributions highly prevalent for subtask 2 and subtask 3 data, the team decided to move forward with using all data for fine

tuning. Since we are fine tuning (not training from scratch), a model that has already been trained on millions of examples, this disparity should not introduce bias into the models.

4 Subtask 1

Subtask 1 was the simplest of all tasks. The objective was to build a simple binary classifier for polarization. The team experimented with three different types of data for training. Those three methods were: (1) standard training data, (2) standard training data though with language id pre-pended to the front of the text, and (3) training using "jumbo dataset". The jumbo dataset consisted of 10,000 training samples per language. The artificial data points were created in a standardized way using a python script and not just querying an LLM for consistency.

4.1 Model Selection

FacebookAI's xlm-RoBERTa-large model from hugging face which can found here <https://huggingface.co/FacebookAI/xlm-roberta-large> was used for all predictions. This model was trained on 2.5 billion data points from 100 different languages. The primary reason for choosing this model was from it's performance in subtask 3. Subtask 3 was completed prior to subtask 1 and as we will see later, subtask 3 implements an ensemble of models to make predictions. Of the chosen models, xlm-RoBERTa-large performed the best.

4.2 Training Processes

Instead of just fine-tuning xlm-RoBERTa-large on the training data provided and making predictions, the team was curious to know the impact that simple data augmentations made to the training set could have on overall model performance. Two simple augmentation techniques were used: pre-pending a language token and training on artificial data. To know the effects, we must have a control group in this experiment and so we fine tuned xlm-RoBERTa-large on the standard training set.

A similar training process was executed when fine tuning each of these three model

variants. The team made their best effort to keep training processes consistent across variants, though due to overall compute power compromises were made when fine tuning xlm-RoBERTa-large using the "jumbo" dataset.

All fine-tuning for this subtask was done using Google Colab to be able to efficiently fine-tune the models using Nvidia L4 Tensor Core GPU's - vastly more efficient than training locally on a Apple M1 Macbook Pro.

The team used a 80/20 training/validation split on the training data provided. In this process the team made sure to stratify the split by language in order to maintain consistent language distributions across the training set and newly created validation set.

To optimize the model performance the team hyper-parameter tuned xlm-RoBERTa-large. This is where the training process varied slightly between the standard training process (1.1), pre-pended language token training process (1.2) and jumbo training set process (1.3). Hyper-parameter tuning 1.3 with the jumbo dataset of 50,000 samples was too expensive, and so since data was text was the standard training data with noise sprinkled in, the model was fine-tuned using the optimal hyper-parameters from the standard model. Hyper-parameter tuning was facilitated using Optuna, an open-source hyper-parameter tuning framework. The sampler used for each of these hyper-parameter tuning jobs was the tree-structured parzen estimator (TPE), the most common and effective sampler offered in the Optuna framework. This framework is essentially a machine learning model that uses two probability density curves to narrow the search space quicker rather than covering an entire grid search. More can be learned here: <https://optuna.readthedocs.io/en/stable/index.html>. Once the optimal hyper-parameters were found for the standard training and pre-pended language token training, a final fine-tuning job was completed using these parameters (optimal standard training params) for jumbo dataset.

Prior to continuing to results, let's discuss more in-depth how the data augmentation techniques were implemented.

4.2.1 Prepend language tokens

This augmentation was simple. Upon loading the training data, the language code ([eng],[spa],[deu],[zho],[arb]) was prepended based on the language. The motivation was to be able to inexpensively add some guardrails for the model to be able to better learn the patterns that are unique by language if it explicitly given the language in the input. Results are discussed below and overall this slightly improved model performance. Here is the code snippet showing how simple this was to implement.

```
for lang in languages:
    # Load train
    train_dfs[lang] = pd.read_csv(data_dir + f"train/{lang}.csv")
    train_dfs[lang]["language"] = lang
    # prepend language token to text col
    train_dfs[lang]["text"] = "[" + lang + " " + train_dfs[lang]["text"]
    .astype(str)
```

4.2.2 Jumbo dataset

The main objective was to create artificial data to grow the size of the dataset to see if leveraging one LLM can help another LLM. Which was covered time and time again in-class, most recently for machine translation when creating a system to translate from a low-resource language to a high-resource language. Though since this was a formal SemEval project, the team decided to take a different route to ensure reproducibility. We created a script that given the original data, adds new data points to the dataset by adding prefixes, suffixes, punctuation and shuffles word order. Example prefixes and suffixes for english are included below.

```
LANG_PREFIXES = {
    "eng": [
        "Honestly, ",
        "Tbh, ",
        "In my opinion, ",
        "Let's be real: ",
        "Hot take: ",
        "Lowkey, ",
    ],
```

```
...
}
LANG_SUFFIXES = {
    "eng": [
        " tbh",
        " fr",
        " lol",
        " smh",
        " imo",
        " honestly"
    ],
    ...
}
```

A similar process is done with punctuation where if a text sample does not end with punctuation, then a punctuation character is added by random selection.

Lastly, a pair of words randomly flip position with eachother to add additional noise. This method is only implemented on English, Spanish and German as this would be counter-intuitive to implement for Arabic and Chinese as there are positional information within their characters. This point is described in section 3.1.

The label distribution of this new dataset remains similar to the original dataset as the new data points are being created by adding noise to the randomly sampled original dataset.

4.3 Results

Only fine-tuning on the jumbo dataset lead to a noticeable bump in performance compared to the standard fine-tuning. Fine-tuning using the pre-pended language tokens made little to no difference.

Fine-tuning	Macro F1	Accuracy
Standard	0.8009	0.8019
Prepend labels	0.8022	0.8029
Jumbo dataset	0.9594	0.9596

Table 3: Subtask 1 training results by fine-tuning technique.

4.3.1 Standard training

The confusion matrix below for the results above in Table 3 is nearly symmetrical. There was not certain class which influenced the evaluation the most. This highlights good training data with minimal bias.

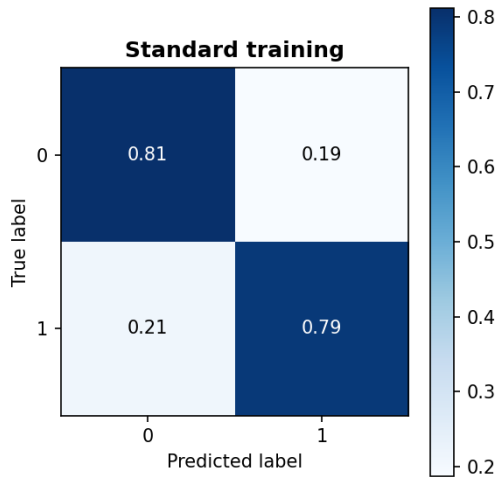


Figure 5: Confusion matrix for standard fine-tuning.

4.3.2 Prepended language tokens

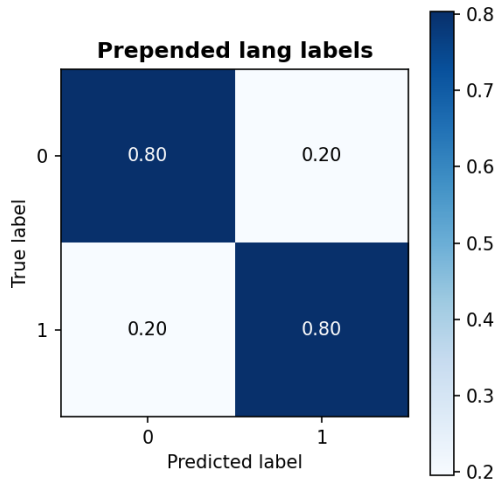


Figure 6: Confusion matrix for fine-tuning with prepended language tokens in training data.

Performance when fine-tuning using the training data with pre-pended language tokens was nearly identical to standard fine tuning above. xlm-RoBERTa-large most likely noticed that every training sample included some variation of the token and ultimately ignored them. Either it knew that high frequency leads to less contextual meaning, such as a document containing 1000's of "the" tokens in it, or since it wasn't originally trained with data of that structure, it ignored them. Since xlm-RoBERTa-large was trained with 2.5B samples from 100 languages it most likely knows deeper patterns between languages than just tokens and so it didn't need them.

4.3.3 Jumbo dataset

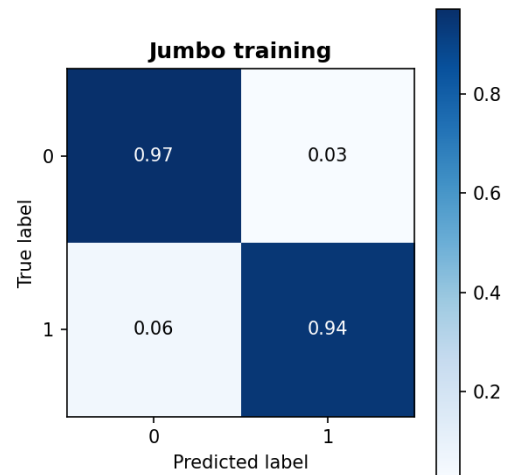


Figure 7: Confusion matrix for fine-tuning with jumbo training dataset.

Fine-tuning using the jumbo dataset lead to phenomenal results. It is unsure whether the extra training time lead to this improvement or the added noise into the training data making the model more robust. This is an interesting point to look into, though wasn't explored by the team due to the vast amounts of compute power required to run such large experiments.

5 Subtask 2

Subtask 2 adds a new layer to the issue of polarization detection by bringing in multi-label classification. Each piece of text may express multiple different emotions or polarizations towards a certain target group or ethnicity. The five categories include: political, racial/ethnic, religious, gender/sexual, and other. These five categories cover a vast range of hostility and polarization among our chosed languages. Unlike Subtask 1, this task will require us to model overlapping social cues, constructs and patterns making it considerably more complex.

To attack this problem, our team built and implemented a multilingual fine-tuning pipeline, using FacebookAI's xlm-RoBERTa-large model (<https://huggingface.co/FacebookAI/xlm-roberta-large>). We chose this model to leverage its cross-lingual representations to hopefully handle the diversity of the data in both language and

language structure. The model was trained on the five languages used in Subtask 1 from the provided starter data set. We did not apply any additional data augmentation or any synthetic generation for Subtask 2. Our focus in Subtask 2 is to develop a foundational baseline detection system and to understand how well a large pretrained model performs in a multi-label classification setting without specific engineering.

5.1 Model Selection

For Subtask 2, we used XLM-RoBERTa-Large, the same encoder for Subtask 1. This model is a 550M-parameter multi-lingual transformer trained on 2.5 billion data points of filtered CommonCrawl data that includes 100 different languages. We configured the model using Hugging Face's `AutoModelForSequenceClassification` specifying:

- **num labels = 5**
- **problem type = "multi label classification"**

These inputs create a classification head that outputs 5 independent logits per one input. Unlike single-label classification, multi-label classification treats each label independently, applying the sigmoid function and limiting each output. We chose this model for similar reasons as Subtask 1. It performed strongly in Subtask 3, in the ensemble, it generalizes well across our five chosen languages, and the nature of Subtask 2 benefits from the models large capacity and multilingual embeddings.

5.2 Training Process

Subtask 1 explored several different data variations, but for Subtask 2, we used a single controlled training set and pipeline to evaluate XLM-RoBERTa-Large's performance on multi-label classification. We combined the five chosen languages into a multi-lingual corpus. Our 80/20 data split was stratified by language to ensure each split preserved the

original language distribution. Our team fine-tuned XLM-RoBERTa-Large using Hugging Face's Trainer framework. In training, we had no data augmentation as our goal was to establish a strong baseline using only the official dataset provided. All experiments used the following hyperparameters:

- **Epochs:** 8
- **Learning Rate:** 2e-5
- **Batch Size:** 16 (train and eval)
- **Weight Decay:** 0.01
- **Warmup Ratio:** 0.1
- **Evaluation Strategy:** Per Epoch
- **Metric for Model Selection:** Macro F1

5.2.1 Label Setup and Compute Considerations

Subtask 2 needs the model to output five independent predictions. After applying the sigmoid to the model logits, our team used a threshold of **0.3** for all labels. This was determined through threshold sweeps during debugging. Lowering our threshold from 0.5 to 0.3 improved recall noticeably for minority labels, leading to an increased F1 macro score. All training was performed in Google Colab, on the T4 tier. The training time was significantly longer than Subtask 1 due to the size of the model chosen and the need to compute multi-label outputs.

5.3 Results

Fine-tuning XLM-RoBERTa-Large for Subtask 2 produced competitive performance across the five polarization categories. Table 4 summarizes the results from the development set.

Performance of the model was strongest with the gender/sexual, racial/ethnic, and political categories. F1 scores for these were around 0.69-0.71. These categories tend to exhibit clearer polarization cues, which could contribute to more accurate learning. The other category was the most difficult for the model

Metric	Score
Macro F1	0.6389
Accuracy	0.6357
Political F1	0.6942
Racial/Ethnic F1	0.6969
Religious F1	0.6325
Gender/Sexual F1	0.7112
Other F1	0.4598

Table 4: Subtask 2 Training Results

to learn, with an F1 of only 0.46. This is reasonable as the other is a broad category likely containing many different types of polarization compared to the other categories, which are very specific. Behavior like this is not uncommon in multi-label classification tasks with a "catch-all" category. Finally, our team noted a gap in the training and validation F1 scores, 0.85 and 0.64, respectively. This suggests overfitting, which is consistent with using a larger model on a smaller amount of multilingual data without any extra data augmentation or engineering. Future work should explore any regularization techniques or data engineering similar to what was used in Subtask 1.

6 Subtask 3

The goal of the subtask 3 was to create a classifier for manifest identification. Similar to subtask 2, this is a multi-class classification model. The data was not equally distributed by language or by class and so the team designed and tested a solution that on paper should be more robust than a standard model.

6.1 Model Selection

The team created a model ensemble for the final classifier for this task. The ensemble consisted of the following models:

- *FacebookAI/xlm-RoBERTa-large*
 - <https://huggingface.co/FacebookAI/xlm-roberta-large>
- *cadiffnlp/twitter-xlm-RoBERTa-base*
 - <https://huggingface.co/cadiffnlp/twitter-xlm-roberta-base>
- *microsoft/mdeberta-v3-base*

– <https://huggingface.co/microsoft/mdeberta-v3-base>

google/rembert, <https://huggingface.co/google/rembert>, was also tested to be in the ensemble although due to the combination of the larger size and poor results it was left out. It would not have made sense to include such a model that yields slightly worse performance compared to the chosen models that also takes longer to train.

Of the models selected, *xlm-RoBERTa-large*, is a known powerhouse of a model and the results held when hyper-parameter tuned. As mentioned above, this model was trained on 2.5B text samples from 100 languages.

twitter-xlm-RoBERTa-base was chosen as it is a condensed architecture of *xlm-RoBERTa-large* though is trained on 198M tweets from different languages. Social media data is noisier than standard data found from crawling official publications on the internet. Our training data is more in line with social media text than standard text and so this was why this model was chosen.

Finally, *microsoft/mdeberta-v3-base* was an experiment. This model uses the *DeBERTa v3* architecture, Disentangled Bidirectional Encoder. Disentangled attention separates contextual and positional info and so the team was interested to see how this model would perform given the classification labels only differed by subtle nuances, i.e. lack of empathy vs vilification vs extreme language.

6.2 Training Processes

Each of these models were fine-tuned the exact way subtask 1 using the *Optuna* hyper-parameter tuning framework. After finding optimal hyper-parameters the models did a final fine-tuning run using those hyper-parameters and those models were saved.

The ensemble is executed by running a data point through each of the 3 trained models, averaging the logits across models and then using the averaged logits to make a decision on class. Additionally, since this is a multi-class classification model, we must choose a certain *threshold* for the model. This parame-

ter represents the min logit value for a certain class to be picked. Since the logits add up to 1, we did not need to worry about this in subtask 1, as there is no reason to change the threshold from 0.5 in this context. Though since we have multiple classes, we must lower the threshold value especially since a single data point can belong to more than just a single class. This threshold tuning step used a simple search from a list of proposed values. After evaluating the training data on each threshold value, the threshold which yielded the highest Macro F1 was chosen. In our case, this value was 0.275.

6.3 Results

Overall, the performance boost from the ensemble model was not as noticeable as expected. Macro-F1, Accuracy and confusion matrices for the performance of each of the individual models the ensemble are provided below.

Model	Macro F1	Accuracy
xlm-RoBERTa-large	0.6920	0.6112
twitter-xlm-RoBERTa-base	0.6814	0.6054
mdeberta-v3-base	0.6533	0.6054
Ensemble	0.7000	0.6207

Table 5: Subtask 3 training results by model.

6.3.1 xlm-RoBERTa-large

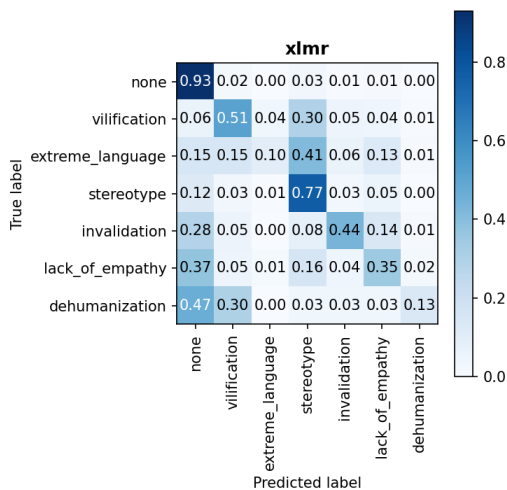


Figure 8: Confusion matrix for xlm-RoBERTa-large on subtask 3.

Though being the best performing individual model of the ensemble, there are still some

prominent short-comings of the results of this model. The most notably the models in ability to recognize 'dehumanization' as polarization alone. When able to recognize 'dehumanization' as polarizing text the model most often labeled it as 'vilification'. The model as struggled with 'extreme language'. Majority of the time the model is labeling 'extreme language' text as 'stereotype' and the only classes that it is actually labeling 'extreme language' as 'extreme language' more than is 'invalidation' and 'dehumanization'. Other than 'none', the model was best at classifying 'stereotype' class.

6.3.2 twitter-xlm-RoBERTa-base

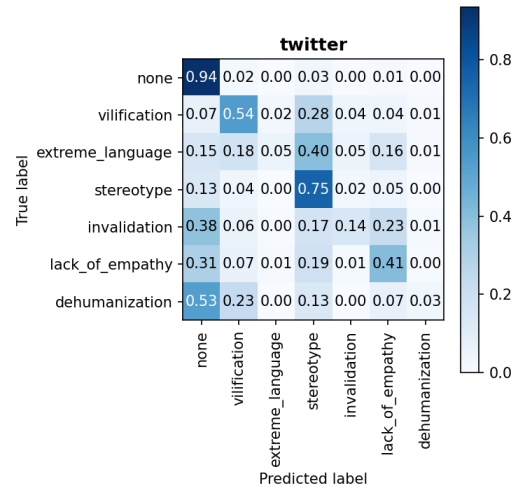


Figure 9: Confusion matrix for twitter-xlm-RoBERTa-base on subtask 3.

Similar short-comings seen in xlm-RoBERTa-large are present here, though performance excels in certain cases. Slight improvements are seen when classifying 'none', 'vilification' and 'lack of empathy'. The most interesting trend in this model is there is a significant performance drop when classifying 'invalidation'. 0.41 in xlm-RoBERTa-large vs 0.14 in twitter-xlm-RoBERTa-base.

6.3.3 mdeberta-v3-base

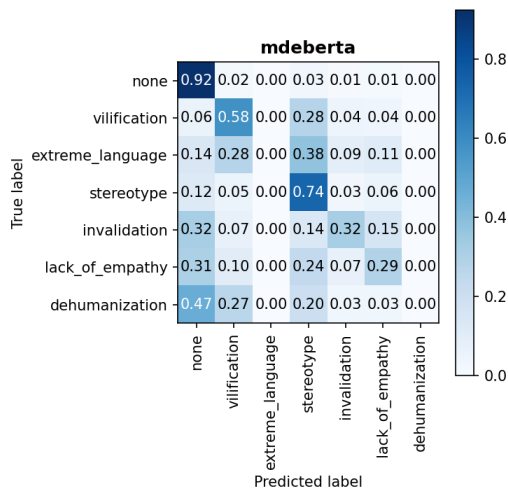


Figure 10: Confusion matrix for mdeberta-v3-base on subtask 3.

Performance trends of this model are most similar to xlm-RoBERTa-large. Overall there are not massive differences in the data, although this model has slightly more errors distributed across the whole gradient which leads to a slight decrease in performance across all classes.

6.3.4 Ensemble

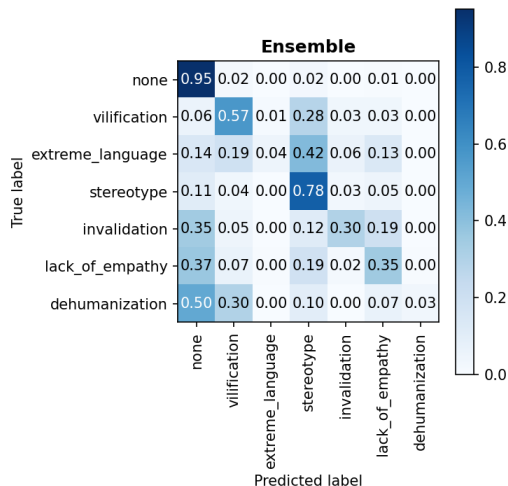


Figure 11: Confusion matrix for ensemble of models on subtask 3.

The ensemble model performed the best of all models and that can be seen above in the confusion matrix. Majority of the classes are predicted just better here than in the other models which is the main culprit for this model

performing the best. The model is not perfect though. The model struggle to correctly identify 'dehumanization' samples more than any other model except for twitter-xlm-RoBERTa-base.

Though the ensemble model out-performed the rest of the models the team agreed that the 300% increase in training time was not worth the small performance boost. Had the team re-done the subtask with the knowledge they have now, they would apply different methods to experiment with how to get the most out of xlm-RoBERTa-large alone.

7 Conclusion

Clever tricks whether that be data augmentations or enhancing the training process have potential to improve performance in LLM's. Though at the end of the day the most noticeable performance bumps occur when implementing bigger model or training the model with more data. This fact first shared in class, was made visible by the work in subtask 1. The team saw the biggest performance boost across the entire project when fine-tuning *xlm-RoBERTa-large* using the artificially created jumbo dataset.